Департамент образования и науки Брянской области
Государственное автономное профессиональное образовательное учреждение
«Новозыбковский профессионально-педагогический колледж»

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА К КУРСОВОМУ ПРОЕКТУ

Разработка веб-приложения для конверсии медиафайлов

Яриго Иван Николаевич
Специальность: 09.02.07
Информационные системы и программирование
III курс, 35 группа
Научный руководитель:
Саросек Сергей Михайлович
преподаватель информатики, спецдисциплин

Новозыбков, 2026

Содержание

Аннотация 3

Введение 5

Глава 1. Теоретические основы асинхронной обработки данных 7

1.1. Понятие асинхронности в веб-приложениях 7

1.2. Очереди задач: брокеры сообщений (RabbitMQ, Apache Kafka) и реляционные базы данных 13

1.3. Принципы работы S3-совместимых объектных хранилищ 19

1.4. Сравнительный анализ существующих решений и обоснование выбора технологий 22

Глава 2. Проектирование и реализация веб-приложения 28

2.1. Архитектура сервиса и взаимодействие компонентов 28

2.2. Проектирование базы данных и объектов хранения 31

2.3. Организация асинхронной обработки задач 35

2.4. Реализация серверной и клиентской частей приложения 37

Заключение 40

Список использованной литературы 42

Аннотация

Данный курсовой проект посвящён разработке асинхронного веб-сервиса для конвертации медиафайлов (изображений, аудио, видео). Целью работы является создание отказоустойчивой системы, позволяющей пользователям загружать файлы, получать идентификатор задачи немедленно и скачивать результат после её обработки.

Пояснительная записка состоит из введения, двух глав, заключения, списка использованной литературы и двух приложений. Общий объём записки составляет 40 страниц.

В первой главе рассматриваются теоретические основы асинхронной обработки данных: дано определение асинхронности в веб-приложениях, выявлены сценарии её применения и ключевые преимущества. Проведён сравнительный анализ способов организации очередей задач на основе брокеров сообщений (RabbitMQ, Apache Kafka) и реляционных баз данных, сформулированы критерии выбора. Изучены принципы работы и архитектура S3-совместимых объектных хранилищ. На основе проведённого анализа обоснован выбор конкретного стека технологий (брокер сообщений, объектное хранилище, язык программирования) для реализации веб-приложения.

Во второй главе описаны проектирование и реализация веб-приложения. Разработана общая архитектура сервиса, определяющая взаимодействие клиентской части, сервера приложений, брокера очередей и S3-совместимого хранилища. Спроектирована схема реляционной базы данных для хранения метаданных задач и пользователей, а также структура бакетов и ключей в объектном хранилище. Реализован механизм асинхронной обработки задач, включающий постановку задач в очередь, их обработку воркерами и отслеживание статуса выполнения с использованием выбранного брокера сообщений. Выполнена реализация серверной части приложения (REST API) и клиентского интерфейса для инициации задач и отображения результатов их выполнения.

Серверная часть приложения разработана на Java 25 с использованием Spring Boot 4, Spring Data JPA. Для организации очереди задач применён PostgreSQL. Интеграция с S3-совместимым хранилищем выполнена через AWS SDK v2. Для конвертации медиафайлов используется FFmpeg через библиотеку JavaCV. Клиентская часть приложения реализована на TypeScript с использованием React, с применением паттерна «конечный автомат» для управления состоянием и периодического опроса статуса задач.

Ключевые слова: асинхронная обработка, очередь задач, брокер сообщений, RabbitMQ, Java, Spring Boot, TypeScript, React, S3-хранилище, многопоточность.

Введение

Современные веб-приложения всё чаще сталкиваются с необходимостью обработки больших объёмов медиаданных – изображений, аудио и видео. Пользователи ожидают быстрого отклика, однако конвертация медиафайлов является ресурсоёмкой и может занимать значительное время. Выполнение таких операций в синхронном режиме приводит к блокировке интерфейса и неэффективному использованию серверных ресурсов. В связи с этим актуальна разработка асинхронных систем, способных принимать задачи на обработку и выполнять их в фоновом режиме.

Объектом исследования: процесс асинхронной обработки медиафайлов в веб-приложениях.

Предмет исследования: методы и средства организации очереди задач, интеграции с объектным хранилищем (S3-совместимым) и реализации веб-интерфейса для управления конвертацией.

Цель работы: разработать отказоустойчивый асинхронный веб-сервис для конвертации медиафайлов, позволяющий пользователям загружать файлы, немедленно получать идентификатор задачи и скачивать результат после завершения фоновой обработки.

Задачи работы:

1. Определить понятие асинхронности в веб-приложениях, выявить сценарии применения и ключевые преимущества.

2. Выполнить сравнительный анализ брокеров сообщений и реляционных баз данных как очередей задач, сформулировать критерии выбора.

3. Изучить принципы работы и архитектуру S3-совместимых хранилищ.

4. Провести сравнительный анализ рассмотренных технологий и обосновать выбор конкретного стека технологий (брокер сообщений, объектное хранилище, язык программирования) для реализации веб-приложения.

5. Разработать общую архитектуру сервиса, определить взаимодействие между клиентской частью, сервером приложений, брокером очередей и S3-совместимым хранилищем.

6. Спроектировать схему реляционной базы данных для хранения метаданных задач и пользователей, а также структуру бакетов и ключей в объектном хранилище для хранения файлов.

7. Реализовать механизм асинхронной обработки задач, включая постановку задач в очередь, их обработку воркерами и отслеживание статуса выполнения с использованием выбранного брокера сообщений.

8. Выполнить реализацию серверной части приложения (API) и клиентского интерфейса для инициации задач и отображения результатов их выполнения.

Методы исследования: анализ научно-технической литературы, сравнительный анализ, проектирование информационных систем, объектно-ориентированное программирование.

Глава 1. Теоретические основы асинхронной обработки данных

1.1. Понятие асинхронности в веб-приложениях

В классической модели взаимодействия компонентов веб-приложения преобладает синхронный подход. При синхронном взаимодействии клиент отправляет запрос серверу и переходит в состояние ожидания ответа. До тех пор, пока сервер не обработает запрос и не вернёт результат, клиент не может продолжить выполнение других операций. Такая модель хорошо знакома по стандартному HTTP-протоколу, где каждый запрос предполагает немедленный ответ [1].

Однако при выполнении длительных операций – отправке электронных писем, генерации отчётов, обработке загруженных изображений или взаимодействии с внешними API – синхронный подход обнаруживает существенные недостатки. Главный из них – неэффективное использование вычислительных ресурсов. Поток выполнения, выделенный для обработки запроса, простаивает в ожидании завершения операции, хотя мог бы быть занят другой полезной работой. В условиях высокой нагрузки это приводит к исчерпанию пула потоков и, как следствие, к отказу в обслуживании новых пользователей. Кроме того, пользователь вынужден наблюдать длительную загрузку страницы, что негативно сказывается на опыте взаимодействия с приложением.

Альтернативой синхронному подходу выступает асинхронное взаимодействие. В этом случае клиент отправляет запрос, но не дожидается его немедленного выполнения. Сервер немедленно подтверждает получение запроса и гарантирует, что задача будет обработана позже. Клиент может продолжать работу, а о результате обработки он узнает дополнительным способом – через повторный запрос, webhook или уведомление по электронной почте. Такая модель аналогична отправке письма: отправитель не стоит у почтового ящика в ожидании ответа, а занимается своими делами [1; 4, с. 390].

Синхронное выполнение длительных операций приводит к неэффективному использованию ресурсов и ухудшению пользовательского опыта. Решением этой проблемы является переход к асинхронной модели взаимодействия, при которой инициатор запроса не дожидается немедленного завершения операции, а продолжает свою работу, полагаясь на то, что задача будет выполнена позже.

Одним из распространённых способов организации асинхронной обработки в веб-приложениях является использование очередей задач. Эта модель предполагает наличие трёх основных компонентов: поставщика задач (producer или publisher), очереди (queue) и темы (topic), и потребителя (consumer или subscriber) [2; 5, с. 6-7]. В отличие от классической событийно-ориентированной архитектуры, где компоненты реагируют на произошедшие события предметной области [3], в модели очередей задач каждое сообщение интерпретируется как явная команда на выполнение конкретной работы (например, «отправить электронное письмо» или «сгенерировать отчёт») [2]. Такой подход проще в реализации и хорошо подходит для задач, требующих гарантированного выполнения и масштабирования обработчиков.

Поставщик задач – это компонент приложения (например, веб-сервер или обработчик API-запроса), который создаёт задачу и помещает её в очередь. Поставщик не занимается непосредственным выполнением задачи, а лишь формирует её описание (часто в виде JSON- или XML-сообщения) и передаёт в промежуточное хранилище. Благодаря этому время ответа пользователю сокращается до минимума [2; 5, с. 6]. Очередь представляет собой буфер, который надёжно хранит поступившие задачи до момента их обработки. Очередь может быть реализована как на основе специализированных брокеров сообщений (RabbitMQ, Apache Kafka), так и с использованием реляционных баз данных. Ключевые свойства очереди – сохранность задач и гарантия их доставки одному из потребителей [2]. Потребитель – это отдельный сервис или процесс (часто называемый воркером), который извлекает задачи из очереди и выполняет предписанные действия. Потребители могут масштабироваться горизонтально:

при увеличении нагрузки достаточно запустить дополнительные экземпляры воркеров, которые будут параллельно забирать задачи из очереди. Такой подход обеспечивает высокую пропускную способность системы [2; 5, с. 7].

В зависимости от требований приложения результат обработки может быть либо проигнорирован (модель «отправил и забыл»), либо записан в постоянное хранилище (базу данных или объектное хранилище) с последующим уведомлением пользователя. Например, после обработки загруженного изображения потребитель может сохранить его в S3-совместимом хранилище и обновить статус задачи в базе данных, что связывает данную модель с тематикой следующих разделов работы.

Несмотря на внешнее сходство, модель очередей задач следует отличать от полноценной событийно-ориентированной архитектуры (Event-Driven Architecture, EDA). В EDA основным объектом передачи являются события предметной области (например, «Заказ оплачен» или «Товар добавлен в корзину»), на которые могут подписываться множество независимых обработчиков, запуская цепочки реакций [3]. В очередях задач сообщение, как правило, адресовано конкретному обработчику и подразумевает выполнение строго определённой работы [2]. Для задач асинхронной обработки данных в веб-приложениях модель очередей является более простой и часто достаточной.

Переход от синхронной модели к асинхронной обработке сопряжён с рядом существенных преимуществ, однако требует также учёта определённых ограничений и архитектурных сложностей. К числу ключевых преимуществ относится повышение отзывчивости пользовательского интерфейса. В синхронных приложениях выполнение длительной операции блокирует поток выполнения, приводя к «заморозке» интерфейса. Асинхронный подход устраняет эту проблему: задача инициируется, но выполняется в фоновом режиме, а поток управления немедленно возвращается к обработке событий интерфейса. Пользователь продолжает работу, а о результате операции он уведомляется позже. Такая модель критически важна для современных одностраничных

приложений (SPA) и мобильных интерфейсов, где плавность взаимодействия напрямую влияет на удовлетворённость пользователя [1; 4, с. 390; 6, с. 3].

Другим важным преимуществом является эффективное использование ресурсов и масштабируемость. При синхронной обработке каждый входящий запрос занимает поток на всё время своего выполнения, включая периоды ожидания ввода-вывода. В это время поток простаивает, не принося пользы. Асинхронная модель позволяет освободить поток во время I/O-ожидания и задействовать его для обработки других запросов. Благодаря этому при том же объёме аппаратных ресурсов сервер способен обслуживать значительно большее количество параллельных операций. Особенно ярко это преимущество проявляется в сочетании с очередями задач. Поставщик задач лишь помещает сообщение в очередь и сразу освобождается, а обработчики работают в своём темпе. Если интенсивность поступления задач временно превышает пропускную способность обработчиков, задачи накапливаются в очереди, но не теряются и не приводят к отказу в обслуживании. При необходимости количество обработчиков можно увеличить горизонтально, что делает систему легко масштабируемой [1; 4, с. 390; 6, с. 3].

Кроме того, асинхронная архитектура повышает надёжность и отказоустойчивость. Использование очередей сообщений обеспечивает сохранность задач даже при сбоях компонентов. Большинство промышленных брокеров поддерживают персистентность: сообщения записываются на диск до того, как будут обработаны. Если обработчик падает во время выполнения задачи, сообщение не теряется – оно возвращается в очередь и может быть обработано другим экземпляром обработчика после перезапуска. Это гарантирует доставку как минимум один раз (at-least-once) и существенно повышает общую надёжность системы. Кроме того, асинхронная архитектура позволяет изолировать сбои. Проблемы в одном из потребителей не влияют на работу поставщиков и на доступность основного приложения для пользователей. Отказ потребителя приводит лишь к временному накоплению задач в очереди, но не к потере данных [7, раздел «Reliability Guide»; 5, с. 161-180].

Вместе с тем асинхронный подход имеет и недостатки. Внедрение асинхронной обработки неизбежно ведёт к появлению дополнительных компонентов: брокеров сообщений, очередей, воркеров, хранилищ результатов. Это усложняет инфраструктуру, требует её администрирования и мониторинга. Сам код с использованием колбэков (callback) или корутин (coroutine) может быть менее очевидным для разработчиков, привыкших к синхронному стилю, хотя современные языковые конструкции (async/await) частично смягчают эту проблему [6, с. 3].

Кроме того, асинхронность не является универсальным решением. Для очень коротких операций, которые выполняются за миллисекунды, накладные расходы на создание задачи, передачу через очередь и управление состоянием могут превысить выгоду от распараллеливания. Асинхронная модель не подходит для задач, требующих немедленного ответа пользователю – например, для проверки подлинности или получения данных для отрисовки страницы. В таких сценариях синхронное взаимодействие остаётся более уместным. Также важно различать I/O-bound и CPU-bound задачи: асинхронность эффективна только для первых, тогда как для вычислительно интенсивных задач лучше использовать многопоточность или распределённые вычисления [8, с. 182, 204].

Проведённый анализ показывает, что асинхронный подход предоставляет значительные преимущества с точки зрения отзывчивости, эффективности использования ресурсов, масштабируемости и надёжности, особенно для задач, связанных с интенсивным вводом-выводом и фоновой обработкой. Однако его внедрение сопряжено с усложнением архитектуры, необходимостью тщательного проектирования и учёта потенциальных проблем согласованности данных. Для задач, подобных рассматриваемой в данной работе (асинхронная обработка пользовательского контента с сохранением в объектное хранилище), преимущества асинхронной модели существенно перевешивают её недостатки, что делает её обоснованным выбором для реализации практической части исследования.

Асинхронная модель взаимодействия применяется в веб-приложениях повсеместно для решения задач, которые невозможно или нерационально выполнять синхронно в потоке пользовательского запроса. Наиболее распространённые сценарии включают отправку электронной почты и уведомлений: формирование и отправка писем через внешние SMTP-серверы или push-сервисы может занимать несколько секунд, асинхронная постановка в очередь позволяет немедленно вернуть пользователю ответ об успешном принятии запроса, гарантируя при этом доставку уведомления. К таким сценариям также относится обработка загружаемых медиафайлов: конвертация видео или аудио в различные форматы являются ресурсоёмкими операциями, выполнение таких задач в фоновых воркерах предотвращает блокировку веб-сервера и позволяет масштабировать обработку в зависимости от нагрузки. Асинхронность широко используется для генерации отчётов и выгрузки документов: создание сложных PDF-отчётов или Excel-таблиц по запросу пользователя часто требует времени, асинхронный подход позволяет поставить задачу в очередь, а затем уведомить пользователя о готовности файла, например, по ссылке для скачивания. Наконец, при взаимодействии с внешними API, если для ответа пользователю необходимо агрегировать данные из нескольких внешних источников, асинхронные HTTP-запросы позволяют выполнять их параллельно, сокращая общее время ожидания [1; 4, с. 390; 8, с. 182, 204].

Все перечисленные сценарии объединяет общая черта: они либо являются длительными (секунды и минуты), либо связаны с ожиданием внешних сервисов (I/O-bound), либо критически важны с точки зрения гарантии доставки. Именно для таких задач асинхронная обработка на основе очередей является стандартом индустрии [1; 4, с. 390; 8, с. 182, 204].

В рамках данной работы в качестве целевого сценария выбрана задача асинхронной обработки пользовательских медиафайлов с сохранением в объектное хранилище, которая обладает всеми перечисленными характеристиками. Это обуславливает выбор очереди задач и S3-совместимого хранилища в качестве ключевых компонентов проектируемого решения.

1.2. Очереди задач: брокеры сообщений (RabbitMQ, Apache Kafka) и реляционные базы данных

Как было показано в пункте 1.1, центральным элементом асинхронной обработки является очередь, в которую поставщики помещают задачи и из которой потребители их извлекают. В простейшем случае функцию такой очереди может выполнять обычная реляционная база данных, уже используемая в приложении для хранения основной информации.

Идея заключается в создании специальной таблицы (например, jobs или tasks), где каждая запись соответствует отдельной задаче. Типичная структура такой таблицы включает идентификатор задачи, её тип, параметры (часто в гибком формате JSON), статус (например, pending, processing, done, failed), время создания и попыток выполнения. Потребители периодически опрашивают таблицу, выбирая задачи со статусом pending, и после успешной обработки обновляют статус на done. Для предотвращения повторного выбора одной задачи разными потребителями используются блокировки на уровне строк (например, SELECT ... FOR UPDATE в PostgreSQL или MySQL) [9].

Основным преимуществом использования реляционной БД в качестве очереди является простота и отсутствие дополнительных инфраструктурных компонентов. Если приложение уже использует базу данных, такой подход не требует развёртывания и администрирования отдельного брокера сообщений. Кроме того, транзакционная модель ACID позволяет гарантировать, что изменение статуса задачи и связанные с ней изменения других данных будут выполнены атомарно – это важно для систем, требующих строгой согласованности.

Однако по мере роста нагрузки и усложнения требований к обработке, у этого подхода проявляются существенные ограничения, которые делают его малопригодным для высоконагруженных и отказоустойчивых систем [9].

Именно для решения этих проблем в современных высоконагруженных системах применяются специализированные программные продукты – брокеры сообщений. Они представляют собой промежуточное ПО, обеспечивающее надёжную асинхронную передачу данных между независимыми компонентами, выступая в роли центрального хаба коммуникации. Рассмотрим архитектуру двух наиболее популярных брокеров: RabbitMQ и Apache Kafka.

Архитектура RabbitMQ базируется на взаимодействии двух основных компонентов: обменников и очередей сообщений. Обменники функционируют как интеллектуальные маршрутизаторы, которые на основе заданных правил направляют поступающие сообщения в соответствующие очереди. Данный механизм обеспечивает архитектурную гибкость: поддержка различных типов обменников позволяет адаптировать систему под широкий спектр сценариев взаимодействия между сервисами. Отличительной особенностью RabbitMQ является его способность работать с распространенными протоколами обмена сообщениями, такими как AMQP, MQTT и STOMP. Это позиционирует брокер как универсальное средство для интеграции гетерогенных систем, требующих реализации сложной логики маршрутизации, а также для приложений, в которых применяются разнообразные паттерны обмена сообщениями [10].

Архитектура Apache Kafka базируется на концепции распределенного журнала сообщений. Данные в Kafka организованы в тематические разделы – топики, которые, в свою очередь, делятся на партиции для обеспечения параллелизма обработки и распределения нагрузки. Все сообщения сохраняются на диске, а их репликация автоматически регулируется между узлами кластера, что обеспечивает высокую степень отказоустойчивости системы.

Kafka демонстрирует наибольшую эффективность в сценариях, требующих обработки потоков данных большой интенсивности: система способна обрабатывать миллионы сообщений в секунду. Ключевым преимуществом данной архитектуры является гарантированное сохранение порядка сообщений в пределах одной партиции, что является критически важным требованием для множества бизнес-процессов. Кроме того, платформа

поддерживает долговременное хранение информации. Указанные характеристики обусловливают применение Kafka преимущественно в высоконагруженных системах с большими объемами данных, а также в приложениях, ориентированных на потоковую обработку информации в режиме реального времени [10].

Каждая из представленных технологий имеет свои особенности и сильные стороны, но как же выбрать между этими тремя решениями на практике? Давайте проведем детальное сравнение по основным параметрам, которые критически важны при выборе брокера сообщений для определенных проектов. В таблице ниже рассматриваются критически важные параметры и сведения для каждого из брокеров.

Результаты сравнительного анализа представлены ниже (см. Таблица 1).

Таблица 1

Сравнительный анализ различных реализаций очереди задач

Критерий	Реляционная база данных как очередь	RabbitMQ	Apache Kafka
Потребление ресурсов сервера	Низкое. Используется уже существующий сервер БД	Среднее [10]. Требуется отдельный процесс/контейнер для брокера. Написан на языке, который использует Erlang VM, что увеличивает	Среднее [10]. Требуется отдельный процесс/контейнер. Написан на языке, который использует JVM, что увеличивает потребление памяти

		потребление памяти	
Необходимость отдельного сервера	Нет. Используется та же СУБД, что и для хранения основных данных	Да. Нужен отдельный брокер	Да. Нужен отдельный кластер Kafka [10]
Время изучения	Низкое. Используется знакомый любому разработчику SQL	Среднее. Требуется устройство RabbitMQ	Среднее. Требуется изучить устройство Kafka
Производительность	Низкая/Средняя. Подходит до ~1,000 задач/с [9]	Средняя. Подходит до ~45,000 задач/с [10]	Очень высокая. Подходит до ~1,000,000 задач/с [10]
Гибкость маршрутизации	Отсутствует. Все задачи кладутся в таблицы, а выборка происходит по статусу. Любая маршрутизация будет самодельной реализацией разработчика [9]	Очень гибкая [10]	Базовая [10]
Устойчивость к сбоям	Зависит от настроек СУБД. При сбое СУБД	Высокая [10]	Высокая [10]

	очередь тоже падает		
Наличие готовых библиотек и инструментов	Есть драйверы для всех языков программирования. В некоторых СУБД существуют различные способы мониторинга очереди (LISTEN/NOTIFY в PostgreSQL) [9]	Доступна на всех основных языках, имеет богатую экосистему [10]	Доступна на всех основных языках, имеет богатую экосистему [10]

Проведённый обзор показывает, что специализированные брокеры сообщений, такие как RabbitMQ и Apache Kafka, предоставляют развитые механизмы маршрутизации, высокую производительность и масштабируемость, необходимые для построения отказоустойчивых высоконагруженных систем. RabbitMQ благодаря гибкой системе обменников и подтверждений хорошо подходит для задач со сложной логикой доставки и требованиями гарантий обработки. Kafka, в свою очередь, обеспечивает колоссальную пропускную способность и долговременное хранение событий, что делает её стандартом для потоковой обработки данных и сбора логов в крупных интернет-сервисах.

Однако применение этих брокеров сопряжено с рядом требований: они нуждаются в развёртывании отдельных сервисов, потребляют дополнительные вычислительные ресурсы (оперативную память, дисковое пространство, процессорное время), а также требуют от разработчика изучения новых парадигм и инструментов. В условиях, когда проектируемая система рассчитана на

невысокую нагрузку (порядка десятков или сотен задач в сутки), а инфраструктура ограничена виртуальным сервером с минимальными характеристиками, использование тяжёлых брокеров может оказаться избыточным и неоправданно ресурсоёмким.

В таких сценариях допустимым и прагматичным решением становится применение уже имеющейся реляционной базы данных (например, PostgreSQL [9]) для организации очереди задач. Хотя такой подход уступает профессиональным брокерам в производительности при высоких нагрузках и не поддерживает сложную маршрутизацию, он обладает рядом преимуществ в контексте малых проектов:

1. Отсутствие дополнительной инфраструктуры. Очередь работает в той же СУБД, что и основное приложение, не требуя установки и администрирования новых компонентов.

2. Минимальное потребление ресурсов. Не запускаются лишние процессы, нагрузка на сервер практически не возрастает по сравнению с обычной работой базы данных.

3. Простота реализации. Разработчик использует знакомый SQL и существующие библиотеки доступа к данным, не тратя время на изучение новых API.

4. ACID-гарантии. Возможность атомарно изменять статус задачи и связанные данные в единой транзакции, что критично для некоторых бизнес-процессов.

Таким образом, выбор между реляционной БД и профессиональным брокером должен определяться контекстом: ожидаемой нагрузкой, доступными ресурсами, сложностью предметной области и квалификацией команды. Для масштабных систем с высокими требованиями к производительности и отказоустойчивости предпочтительны RabbitMQ или Kafka. Для небольших проектов, работающих в условиях ограниченных ресурсов и не предъявляющих жёстких требований к пропускной способности, использование реляционной базы данных в качестве очереди является вполне оправданным решением,

позволяющим сосредоточиться на реализации бизнес-логики без излишних инфраструктурных затрат.

1.3. Принципы работы S3-совместимых объектных хранилищ

После рассмотрения подходов к асинхронной обработке данных возникает необходимость решить проблему хранения: все исходные файлы пользователей и результаты конверсии нужно где-то сохранять. Традиционные методы, такие как хранение на диске сервера или в сетевых файловых системах, а также использование реляционных баз данных для BLOB, неудобны из-за ограничений масштабируемости, отказоустойчивости и резервного копирования.

Современный подход – объектные хранилища, предназначенные для хранения больших объёмов неструктурированных данных с доступом по HTTP/HTTPS, высокой масштабируемостью и встроенной репликацией. Протокол S3 стал де-факто стандартом, а открытые S3-совместимые решения вроде MinIO, Ceph, SeaweedFS и Garage позволяют эффективно организовать хранение как исходных данных, так и результатов асинхронной обработки.

В отличие от иерархической файловой системы, объектное хранилище оперирует тремя основными понятиями:

1. Объект (object) – набор данных, сопровождаемый метаданными (тип, размер, дата создания, пользовательские атрибуты) и уникальным идентификатором (ключом) [11, с. 72-75].

2. Бакет (bucket) – логический контейнер для объектов. Бакет подобен папке, но не поддерживает вложенность на уровне архитектуры [11, с. 61-64].

3. Ключ (key) – уникальное имя объекта внутри бакета. Пространство имён является плоским, однако часто ключи формируются по схеме с разделителями (например, users/123/avatar.jpg), что имитирует иерархию для удобства работы [11, с. 72].

Доступ к объектам осуществляется через REST API с использованием стандартных HTTP методов: PUT – загрузка объекта, GET – получение объекта, DELETE – удаление [11, 75-109]. Метаданные передаются в HTTP-заголовках. Такая модель позволяет легко интегрировать хранилище с веб-приложениями, CDN и любыми инструментами, поддерживающими HTTP.

Фактическим стандартом для объектных хранилищ стал протокол Amazon S3 (Simple Storage Service), предложенный Amazon Web Services. Спецификация его API со временем была открыта, что позволило множеству производителей и open-source сообществ создавать совместимые реализации. Благодаря этому разработчики могут использовать единые клиентские библиотеки, инструменты командной строки (например, AWS CLI) и легко мигрировать между различными поставщиками.

В открытом программном обеспечении существуют несколько зрелых S3-совместимых решений, каждое из которых имеет свои особенности.

1. MinIO – наиболее популярное решение для самостоятельного развёртывания. Написано на языке Go, отличается высокой производительностью, простотой установки (один исполняемый файл, поддержка Docker и Kubernetes). Хорошо подходит для разработки, тестирования и эксплуатации в условиях ограниченных ресурсов.

2. Ceph – мощная распределённая платформа, предоставляющая не только объектное, но также блочное и файловое хранилища. Отличается высокой надёжностью и масштабируемостью, но требует глубоких знаний для установки и сопровождения. Применяется в крупных облачных инфраструктурах.

3. SeaweedFS – легковесная распределённая файловая система, оптимизированная для хранения очень большого количества маленьких файлов (миллиардов). Имеет S3-совместимый шлюз. Проще в настройке, чем Ceph, но функционально более ограничена.

4. Garage – сравнительно новое решение, спроектированное с фокусом на мультидатацентровую репликацию и простоту развёртывания. Позиционируется как компактная альтернатива для edge-сред и децентрализованных систем.

В контексте асинхронной обработки объектное хранилище выступает в роли общего пространства для обмена данными между компонентами системы. Типовой сценарий выглядит следующим образом:

1. Поставщик – например, веб-сервер, принимающий файлы от пользователя, – загружает исходный файл в бакет и получает в ответ ключ объекта.

2. Поставщик формирует задачу, включающую в себя ключ объекта и необходимые параметры обработки, и помещает её в очередь (рассмотренную в разделе 1.2).

3. Потребитель – воркер – извлекает задачу из очереди, по ключу загружает файл из объектного хранилища, выполняет требуемые преобразования и сохраняет результат обратно в хранилище (возможно, в другой бакет или с новым ключом).

4. После завершения обработки результат становится доступным для выдачи пользователю либо через прямую ссылку на объект, либо через дополнительный сервис.

Такой подход имеет ряд преимуществ:

1. Разгрузка очереди – вместо передачи больших бинарных данных через брокер сообщений передаётся только идентификатор объекта (например, строка ключа).

2. Разделение ответственности – компоненты, работающие с файлами, и компоненты, управляющие очередями, могут масштабироваться независимо.

3. Простота интеграции – единый S3 API позволяет использовать одни и те же клиентские библиотеки на всех этапах обработки.

Рассмотренные S3-совместимые хранилища различаются по архитектуре, сложности администрирования, потреблению ресурсов и областям применения. Детальное сравнение их характеристик, а также обоснование выбора конкретного продукта для проектируемой системы будет выполнено в пункте 1.4. Уже на данном этапе можно отметить, что для малых и средних проектов с ограниченными ресурсами часто предпочтительным оказывается MinIO,

сочетающий высокую производительность с минимальными затратами на эксплуатацию.

1.4. Сравнительный анализ существующих решений и обоснование выбора технологий

В предыдущих разделах были рассмотрены различные подходы к организации очередей задач (реляционные базы данных, RabbitMQ, Apache Kafka) и объектные хранилища, совместимые с S3 (MinIO, Ceph, SeaweedFS, Garage). Цель данного раздела – на основе выявленных характеристик сформулировать технологический стек, который будет использован в практической части работы.

Сравнение проводится по следующим критериям, отражающим реальные условия проектирования:

1. Потребление ресурсов (оперативная память, процессор, дисковое пространство) – критически важно, так как система разворачивается на виртуальном сервере с минимальными характеристиками (1 vCPU, 1–2 ГБ RAM).

2. Простота развёртывания и администрирования – возможность установки и поддержки без привлечения дополнительного администратора.

3. Наличие опыта у разработчика – минимизация времени на изучение новых технологий.

4. Достаточная производительность – ожидаемая нагрузка невысока (сотни задач в сутки), поэтому сверхвысокая пропускная способность не требуется.

5. Возможность интеграции с остальными компонентами системы (веб-приложение, обработчики задач).

После выбора объектного хранилища возникает вопрос организации самой асинхронной обработки. Файлы могут храниться где угодно, но для их обработки нужна очередь задач, через которую компоненты системы смогут обмениваться информацией о новых или уже обработанных объектах. На этом этапе важно

определить технологию очереди, учитывая ограничения ресурсов, простоту эксплуатации и ожидаемую нагрузку.

Реляционная база данных как очередь. Использование PostgreSQL (или другой реляционной СУБД) для хранения задач требует лишь создания дополнительной таблицы. Такой подход:

1. Не вносит новых инфраструктурных компонентов – СУБД уже используется для основных данных.

2. Потребляет минимальные ресурсы: дополнительные индексы и несколько десятков мегабайт памяти.

3. Позволяет использовать знакомый SQL, что снижает порог входа.

4. Гарантирует ACID-транзакции, что важно при необходимости атомарно менять статус задачи вместе с другими данными.

Однако реляционная БД не предназначена для высоконагруженных очередей: при интенсивном опросе таблицы могут возникать блокировки, и пропускная способность ограничена. Для нашего проекта с низкой ожидаемой нагрузкой это некритично.

RabbitMQ. Специализированный брокер сообщений на основе протокола AMQP. Предоставляет гибкую маршрутизацию через обменники, гарантии доставки, удобный веб-интерфейс. Однако:

1. Требует установки и поддержки отдельного сервиса (Erlang VM, менеджмент-узел).

2. Потребляет дополнительные ресурсы (минимум 200–300 МБ RAM).

3. Требует времени на изучение концепций (обменники, привязки, подтверждения).

Apache Kafka. Платформа для потоковой обработки данных, построенная на распределённом логе. Обеспечивает колоссальную пропускную способность, долговременное хранение сообщений, горизонтальное масштабирование. Но:

1. Для отказоустойчивости обычно рекомендуется кластер из трёх узлов.

2. Потребляет значительные ресурсы (JVM, минимум 4–6 ГБ RAM для продуктивного использования).

3. Сложность настройки и администрирования высока, требует глубокого понимания внутренних механизмов.

Для нашего проекта, разворачиваемого на слабом виртуальном сервере, использование RabbitMQ или Kafka привело бы к неоправданному усложнению инфраструктуры и повышенному потреблению ресурсов. PostgreSQL же встраивается в существующее окружение без дополнительных затрат.

Рассмотрев варианты реализации очередей задач, можно перейти к анализу хранения больших данных. Даже при эффективной организации очереди все исходные файлы и результаты конверсии необходимо сохранять. На фоне ограничений реляционных БД и локальных файловых систем объектные хранилища, совместимые с S3, представляют собой более подходящее решение. Они позволяют надежно хранить данные, масштабироваться при необходимости и легко интегрироваться с очередями и веб-приложением.

MinIO – наиболее простое в установке S3-совместимое хранилище. Запускается одним исполняемым файлом или контейнером, потребляет мало памяти (порядка 100–200 МБ), предоставляет встроенную консоль и полностью совместим с S3 API. Широкая документация и активное сообщество.

Ceph – мощная распределённая система, поддерживающая объектное, блочное и файловое хранилища. Требует развёртывания нескольких узлов, сложна в настройке, потребляет много ресурсов (десятки гигабайт RAM в кластере). Применяется в крупных облачных инфраструктурах, для нашего проекта избыточна.

SeaweedFS – легковесная распределённая файловая система, оптимизированная для большого числа мелких файлов. Имеет S3-совместимый шлюз, но требует дополнительной настройки. Может работать на ограниченных ресурсах, однако её экосистема и документация менее развиты, чем у MinIO.

Garage – современное решение с фокусом на мультидатацентровую репликацию. Позиционируется как компактная альтернатива, но пока не имеет столь широкого распространения, как MinIO, и требует более глубокого изучения.

С учётом требований к простоте развёртывания, минимальному потреблению ресурсов и наличию у автора опыта работы с MinIO (а также широкой распространённости этого решения в учебных и небольших проектах), выбор очевиден.

Сравнение популярных S3-совместимых хранилищ показывает, что для малого проекта с ограниченными ресурсами и знакомым автору стеком оптимальным выбором является MinIO. Это решение сочетает легкость установки, минимальное потребление ресурсов и удобный интерфейс, что позволяет сконцентрироваться на разработке бизнес-логики, не отвлекаясь на сложное администрирование. На основе проведённого анализа принимается следующее архитектурное решение:

Очередь задач – реализуется на базе PostgreSQL с помощью отдельной таблицы conversion_jobs. Такой подход позволяет:

- избежать установки и поддержки дополнительных сервисов;
- использовать существующий сервер базы данных, что критично при ограниченных ресурсах;
- обеспечить атомарность операций (например, при создании задачи вместе с другими изменениями в транзакции);
- быстро разработать и отладить логику на знакомом SQL.

Объектное хранилище – используется MinIO, развёрнутое в контейнере Docker на том же сервере. Выбор обусловлен:

- минимальным потреблением ресурсов (легковесный Go-бинарный файл);
- полной совместимостью с S3 API, что позволяет использовать стандартные клиентские библиотеки;
- простотой установки и настройки;
- наличием удобного веб-интерфейса для мониторинга;
- опытом автора в работе с этим продуктом.

Оба компонента вписываются в лимиты доступного сервера (1 vCPU, 1–2 ГБ RAM) и не требуют сложного администрирования. При этом они обеспечивают необходимый функционал для реализации сценария асинхронной обработки данных.

После выбора компонентов для очереди задач и объектного хранения возникает необходимость определить инструменты разработки для реализации серверной и клиентской частей системы. Выбранные языки программирования и фреймворки должны обеспечивать удобную интеграцию с PostgreSQL и MinIO, поддерживать многопоточность и REST API на сервере, а на клиенте – интерактивное управление состоянием задач и удобный пользовательский интерфейс. При этом предпочтение отдаётся технологиям с богатой документацией и активным сообществом, чтобы ускорить процесс разработки и снизить время на решение возникающих проблем.

Для реализации серверной части выбран Java 25 с использованием Spring Boot 4. Данный выбор обусловлен наличием встроенных средств для многопоточной обработки (ThreadPoolTaskExecutor), удобной интеграции с PostgreSQL через Spring Data JPA, а также богатой экосистемы для работы с S3-совместимыми хранилищами (AWS SDK v2). Spring Boot также предоставляет готовые механизмы для создания REST API и управления зависимостями, что ускоряет разработку.

Клиентская часть реализована на TypeScript с использованием React; применение конечного автомата для управления состоянием позволяет удобно реализовать периодический опрос статуса задач.

Для упрощения развёртывания и обеспечения изоляции всех компонентов используется Docker (контейнеризация) и Docker Compose, а для обратного проксирования и автоматического получения TLS-сертификатов – Caddy.

Выбранные инструменты соответствуют целям проекта, обеспечивают высокую производительность и масштабируемость, а также позволяют сосредоточиться на реализации бизнес-логики.

На основе проделанного анализа можно сформулировать архитектурное решение для проекта. Выбор PostgreSQL для очереди задач и MinIO для хранения объектов обеспечивает баланс между простотой внедрения, экономией ресурсов и необходимой функциональностью. Выбор Java для сервера и TypeScript для клиента обеспечивает скорость и простоту разработки. Такой стек позволяет сразу перейти к практической реализации системы, описанной во второй главе.

На основе сравнительного анализа выбран технологический стек: PostgreSQL для очереди задач и MinIO для объектного хранения, что полностью соответствует условиям проектирования.

Таким образом, теоретические основы, заложенные в первой главе, позволяют перейти к практической разработке – проектированию и реализации системы асинхронной обработки данных, которая будет описана во второй главе.

Глава 2. Проектирование и реализация веб-приложения

2.1. Архитектура сервиса и взаимодействие компонентов

На основе теоретических выводов, сделанных в первой главе, разработана архитектура веб-сервиса для асинхронной конвертации медиафайлов. Система состоит из пяти основных компонентов: клиентского приложения (React SPA), reverse proxy (Caddy), серверного приложения (Spring Boot), реляционной базы данных (PostgreSQL) и объектного хранилища (MinIO). Рисунок показывает общую схему компонентов и их взаимосвязи (см. Рис. 1)

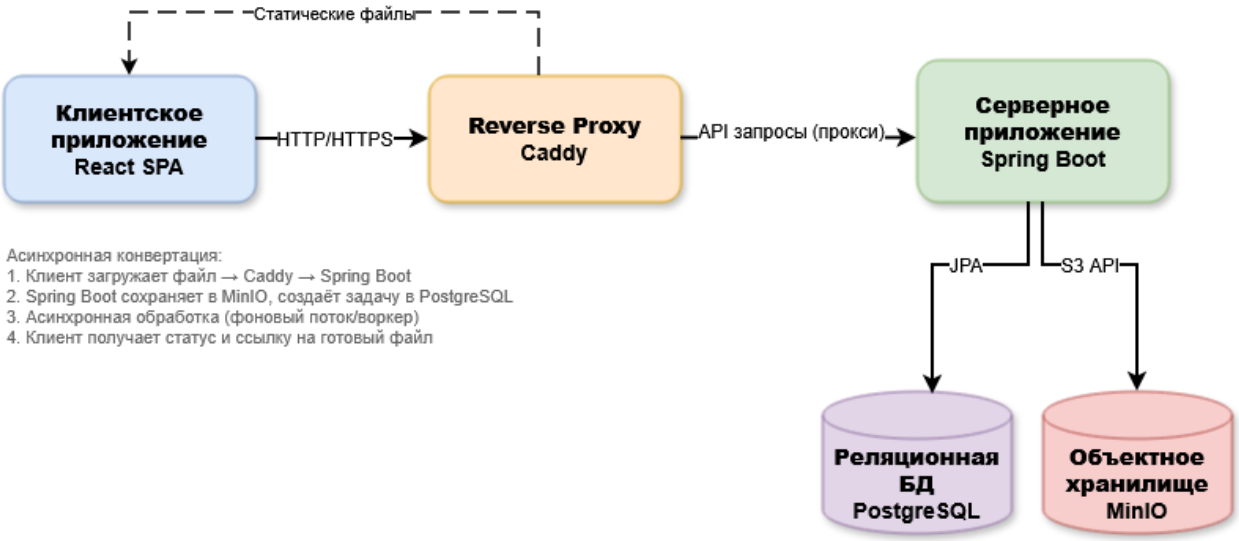

Рисунок 1. Архитектура веб-сервиса для асинхронной конвертации медиафайлов

Клиентское приложение реализовано на TypeScript с использованием React. Оно отвечает за загрузку файлов, отображение статусов задач и скачивание результатов. Reverse proxy на базе Caddy выполняет две функции: маршрутизацию HTTP-запросов между клиентом и сервером, а также автоматическое получение и обновление TLS-сертификатов от Let’s Encrypt. Серверное приложение на Spring Boot содержит бизнес-логику: создание задач, управление очередью, конвертацию через FFmpeg и взаимодействие с хранилищами. PostgreSQL используется для хранения метаданных задач

(таблица conversion_jobs), а MinIO – для хранения исходных и преобразованных файлов.

Функциональные требования, предъявляемые к системе, включают:

- загрузку медиафайла с указанием типа конвертации;
- немедленное получение уникального идентификатора задачи;
- возможность отслеживать статус задачи по идентификатору;
- получение списка задач пользователя (групповой опрос);
- скачивание результата после успешной конвертации.

Схема взаимодействия при создании и обработке задачи проиллюстрирована на рисунке (см. Рис. 2).

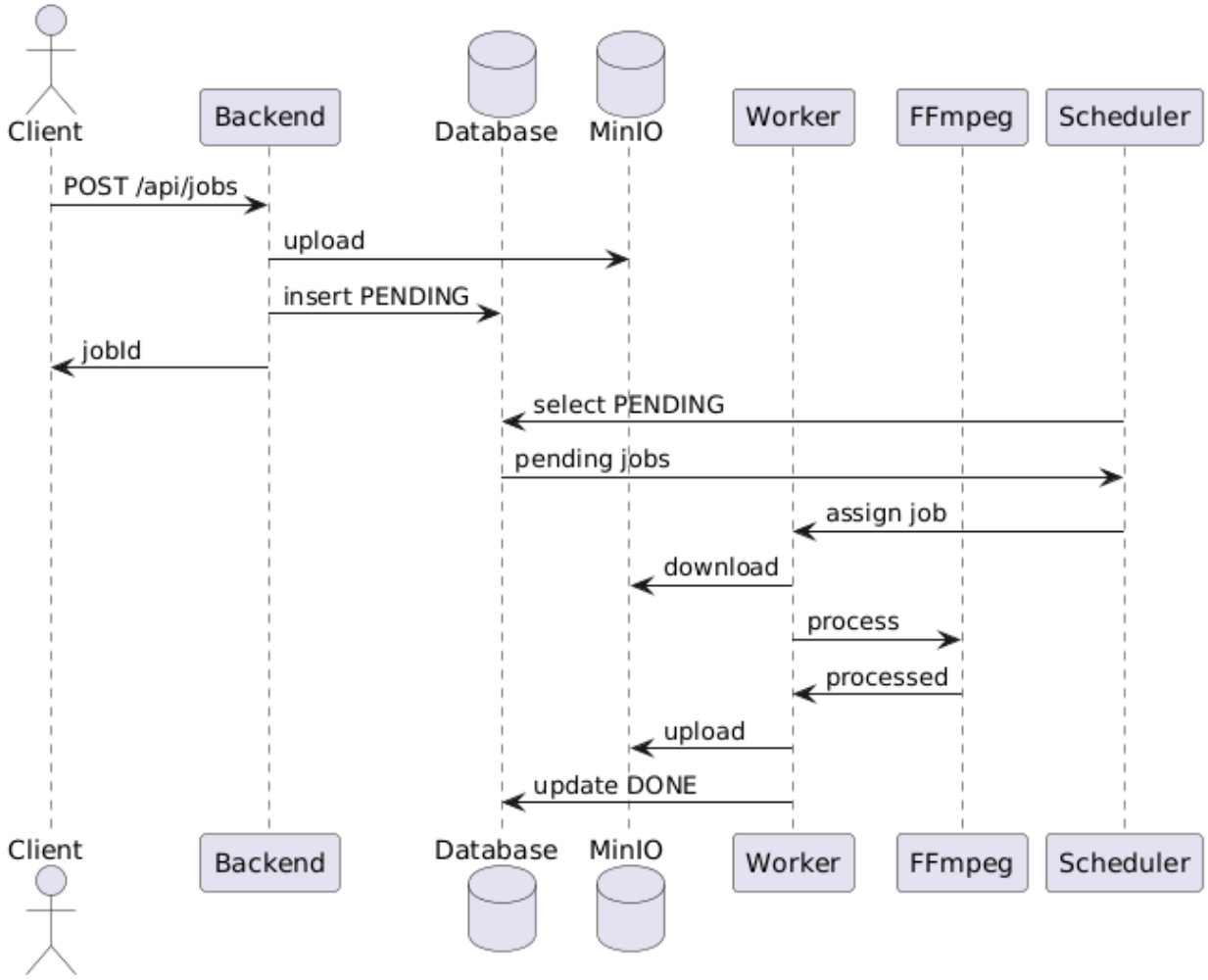

Рисунок 2. Схема взаимодействия при создании и обработке задачи

Процесс начинается с того, что пользователь отправляет файл и параметры конвертации через клиент. Серверное приложение загружает файл в MinIO в бакет conversions, используя ключ, сформированный методом KeyGenerator.inputKey(jobId, originalFilename) – то есть вида {jobId}/input/{санитизированное_оригинальное_имя_файла} (недопустимые символы заменяются на _). После этого создаётся запись в таблице conversion_jobs со статусом PENDING, а клиенту возвращается jobId. Клиент начинает периодически опрашивать эндпоинт /api/jobs/{id}.

На стороне сервера работает планировщик Spring (@Scheduled), который каждые 5 секунд запрашивает из базы данных задачи со статусом PENDING (с использованием SELECT ... FOR UPDATE SKIP LOCKED для предотвращения дублирования) и передаёт их в пул воркеров. Пул воркеров настроен через ThreadPoolTaskExecutor с фиксированным числом потоков, что позволяет контролировать нагрузку на сервер. Каждый воркер загружает исходный файл из MinIO, запускает конвертацию через FFmpeg (обёртка JavaCV), а результат сохраняет обратно в MinIO, после чего обновляет статус задачи в БД на `DONE` (или `FAILED` в случае ошибки). При успешной обработке для сохранения результата конвертации будет использоваться KeyGenerator.outputKey(jobId, outputFormat), формирующий ключ вида {jobId}/output/converted.{outputFormat}.

Обоснование принятых архитектурных решений подробно приведено в первой главе, но здесь следует повторить ключевые аргументы. Использование PostgreSQL в качестве очереди вместо специализированных брокеров (RabbitMQ, Kafka) обусловлено ограниченными ресурсами сервера (1 vCPU, 1–2 ГБ RAM), невысокой ожидаемой нагрузкой (сотни задач в сутки) и желанием избежать дополнительной инфраструктуры. Двухуровневая очередь (таблица БД + планировщик + пул воркеров) обеспечивает персистентность задач и защиту от перегрузки. Пул с ограниченным числом потоков предотвращает истощение ресурсов при одновременной обработке нескольких тяжёлых файлов.

Таким образом, предложенная архитектура полностью соответствует целям проекта, обеспечивая асинхронную обработку медиафайлов в условиях

ограниченных ресурсов. Детальное проектирование базы данных и объектного хранилища будет представлено в пункте 2.2.

2.2. Проектирование базы данных и объектов хранения

Центральным элементом хранения метаинформации о задачах является реляционная база данных PostgreSQL. Основная сущность – таблица conversion_jobs, схема которой представлена в таблице 2.

Таблица 2

Схема таблицы conversion_jobs

Поле	Тип данных	Ограничения	Описание
id	UUID	PRIMARY KEY, NOT NULL	Первичный ключ, генерируется при создании задачи
filename	TEXT	NOT NULL	Имя загруженного файла
input_s3_key	TEXT	NOT NULL	Ключ исходного файла в MinIO (например, {jobId}/input/photo.jpg)
output_s3_key	TEXT		Ключ результата конвертации (заполняется после успешной обработки)
input_format	media_format (ENUM)	NOT NULL	Медиаформат исходного файла
output_format	media_format (ENUM)	NOT NULL	Медиаформат результата конверсии

status	job_status (ENUM)	NOT NULL	Текущий статус задачи. По умолчанию PENDING
error_message	TEXT		Текст ошибки для логирования
created_at	TIMESTAMPTZ	NOT NULL	Время и дата создания задачи
processed_at	TIMESTAMPTZ		Время и дата окончания обработки задачи

Первичный ключ выбран типа UUID, что позволяет генерировать идентификаторы на стороне приложения без обращения к базе данных, упрощая масштабирование. Статус задачи и медиаформаты представлены отдельными ENUM типами.

Для обеспечения производительности выборки задач, готовых к обработке, создаются индексы. Основной индекс – по полю status и created_at, ускоряющий запросы планировщика:

CREATE INDEX idx_status_created ON conversion_jobs(status, created_at);

Частичный индекс для статуса `PENDING` может быть полезен при большом количестве открытых задач, но в условиях невысокой нагрузки достаточно общего индекса.

После создания таблицы и индексов необходимо определить, каким образом PostgreSQL будет выполнять роль очереди задач. В отличие от специализированных брокеров сообщений, реляционная база данных не имеет встроенных механизмов «вытолкнуть» задачу потребителю, однако с помощью правильно составленного SQL-запроса можно атомарно выбрать, заблокировать и изменить статус задачи, готовой к обработке.

Чтобы избежать состояния гонки и гарантировать, что каждая задача будет обработана только одним воркером, используется механизм SELECT ... FOR

UPDATE SKIP LOCKED. Весь процесс захвата задачи выполняется одним запросом (см. Рис. 3).

Захват задачи

```
UPDATE conversion_jobs
SET status = 'PROCESSING'
WHERE id IN (
  SELECT id
  FROM conversion_jobs
  WHERE status = 'PENDING'
  FOR UPDATE SKIP LOCKED
  LIMIT :limit
)
RETURNING *;
```

Рисунок 3. Фрагмент кода запроса захвата задачи

Когда планировщик Spring (@Scheduled) отправляет этот запрос базе данных, PostgreSQL выполняет следующие действия:

1. находит id задач со статусом `PENDING`;
2. блокирует найденные строки (FOR UPDATE SKIP LOCKED), пропуская уже заблокированные другими транзакциями;
3. обновляет статус этих задач на PROCESSING;
4. возвращает приложению полные записи изменённых задач.

Таким образом, вся логика конкурентной блокировки, выборки и обновления статуса выполняется внутри СУБД. Приложение получает только готовые задачи для обработки, что полностью исключает состояние гонки и упрощает код.

Следующим важным компонентом является объектное хранилище MinIO. Для хранения файлов создаётся отдельный бакет conversions. Внутри бакета используется плоская структура ключей с префиксами, имитирующая иерархию.

Ключи формируются с помощью вспомогательного класса KeyGenerator (см. Рис. 4).

KeyGenerator

```
package ru.yarigo.mediaconversionservice.job.service;

import java.util.UUID;

public class KeyGenerator {
    public static String inputKey(UUID jobId, String originalFilename) {
        return jobId + "/input/" + sanitize(originalFilename);
    }

    public static String outputKey(UUID jobId, String outputFormat) {
        return jobId + "/output/converted." + outputFormat.toLowerCase();
    }

    private static String sanitize(String filename) {
        return filename.replaceAll("[^a-zA-Z0-9._-]", "_");
    }
}
```

Рисунок. 4. Фрагмент кода вспомогательного класса KeyGenerator

Входной файл для задачи сохраняется с ключом вида {jobId}/input/{санитизированное_оригинальное_имя_файла}, а результат конвертации помещается в {jobId}/output/converted.{outputFormat}. Такой подход обеспечивает уникальность файлов по каждой задаче и упрощает их последующее управление.

Такой подход обеспечивает уникальность ключей и позволяет легко удалять все файлы, связанные с задачей, по префиксу {jobId}/. Метаданные файлов (оригинальное имя, MIME-тип, размер) могут храниться как пользовательские метаданные S3, что снижает нагрузку на базу данных.

Таким образом, спроектированная схема базы данных и структура объектного хранилища обеспечивают эффективное хранение метаданных и файлов, а также поддерживают требуемую асинхронную обработку задач.

2.3. Организация асинхронной обработки задач

Разработанная в пункте 2.2 схема таблицы conversion_jobs и механизм атомарного захвата задач с помощью UPDATE … FOR UPDATE SKIP LOCKED создают основу для реализации асинхронной обработки. Однако для того, чтобы система могла автоматически находить новые задачи, распределять их между воркерами и управлять ресурсами, необходимы дополнительные компоненты на уровне приложения. В данном разделе описывается организация двухуровневой очереди: планировщик, периодически запускающий захват задач, и пул воркеров с ограниченным числом потоков, непосредственно выполняющий конвертацию (см. Рис. 5).

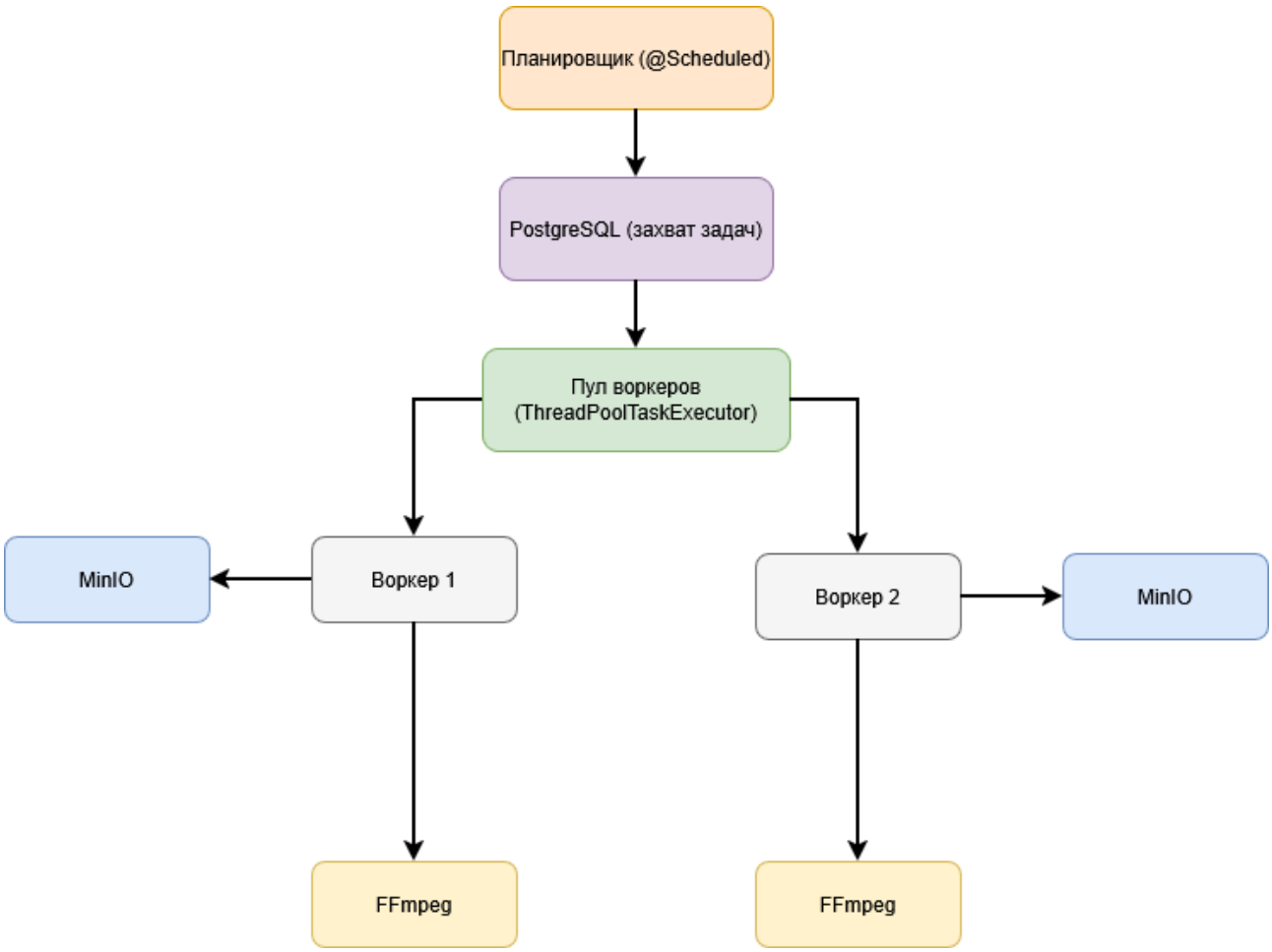

Рисунок 5. Общая схема работы планировщика и пула воркеров

Планировщик Spring, аннотированный @Scheduled, запускается с фиксированной задержкой (например, 5 секунд). В каждом цикле он выполняет

SQL-запрос захвата задач, получая ограниченное число задач, уже переведённых в статус PROCESSING. Для каждой полученной задачи планировщик формирует задание и передаёт его в пул воркеров. Такая конструкция позволяет отделить механизм извлечения задач от их выполнения, обеспечивая возможность горизонтального масштабирования – достаточно запустить несколько экземпляров приложения, и они будут конкурировать за задачи через блокировки базы данных.

Пул воркеров реализован на основе ThreadPoolTaskExecutor. Его настройки подобраны исходя из ограниченных ресурсов сервера (1 vCPU, 1–2 ГБ RAM):

1. corePoolSize: 2 (минимальное количество постоянно активных потоков).

2. maxPoolSize: 4 (максимальное количество потоков, создаваемых при заполнении очереди).

3. queueCapacity: 10 (размер очереди заданий, ожидающих выполнения).

Такая конфигурация позволяет одновременно обрабатывать не более четырёх задач, предотвращая исчерпание памяти и перегрузку процессора. Если все потоки заняты и очередь заполнена, новые задания, переданные планировщиком, будут выполнены самим планировщиком, благодаря политике отказа CallerRunsPolicy.

Каждый воркер, получив свою задачу, сразу приступает к ей обработке. Логика обработки реализована по следующему алгоритму:

1. Скачать файл из хранилища во временное место.

2. Передать файлы и параметры методу конверсии.

3. Если конверсия выполнена успешно: загрузить результат в хранилище, установить статус задачи на DONE и сохранить output_s3_key. В противном случае: установить статус задачи на FAILED и зафиксировать error_message.

Таким образом, разработанная двухуровневая очередь (планировщик + пул воркеров) в сочетании с атомарным захватом задач на уровне СУБД обеспечивает надёжную асинхронную обработку в условиях ограниченных ресурсов. Все компоненты спроектированы таким образом, чтобы их можно было легко масштабировать горизонтально при необходимости – достаточно запустить

несколько экземпляров приложения, которые будут конкурировать за задачи через блокировки базы данных.

2.4. Реализация серверной и клиентской частей приложения

На основе спроектированной архитектуры и механизмов асинхронной обработки были разработаны серверная часть на Java (Spring Boot) и клиентская часть на TypeScript (React). Реализация ориентирована на выполнение всех функциональных требований, сформулированных в пункте 2.1, и учитывает ограниченные ресурсы целевого сервера.

Серверная часть построена на Spring Boot 4 с использованием Spring Data JPA, Spring Web и AWS SDK v2 для работы с MinIO. JPA-сущность ConversionJob отображает таблицу conversion_jobs, описанную в пункте 2.2. Статус реализован в виде перечисления JobStatus (PENDING, PROCESSING, DONE, FAILED). Для атомарного захвата задач в репозитории объявлен метод findByStatus, использующий нативный запрос UPDATE ... FOR UPDATE SKIP LOCKED (см. Рис. 3). Репозиторий наследует JpaRepository, что обеспечивает базовые CRUD-операции.

Сервисный слой представлен классами JobService, StorageService, ConversionService и ValidationService. JobService реализует создание задачи: принимает файл и параметры конвертации, загружает файл в MinIO через StorageService, формирует ключи с помощью KeyGenerator, сохраняет запись в БД со статусом PENDING и возвращает сгенерированный jobId. При создании выполняется валидация типа файла через ValidationService. StorageService инкапсулирует вызовы AWS SDK v2: методы putObject, getObject, deleteObject. ConversionService управляет выбором и запуском стратегии конвертации на основе её параметров. ValidationService управляет выбором и запуском стратегии валидации на основе переданного формата.

Глобальный обработчик исключений @ControllerAdvice преобразует все ошибки в единообразный JSON-ответ с кодом HTTP и сообщением.

Реализация планировщика и пула воркеров (пункт 2.3) также входит в серверную часть: класс PostgresJobConsumer с аннотацией @Scheduled вызывает findByStatus и передаёт задачи в ThreadPoolTaskExecutor, сконфигурированный как бин.

REST-контроллер JobController предоставляет эндпоинты:

1. POST /api/v2/jobs: принимает multipart/form-data (файл и параметр conversionType), возвращает { jobId, status }.

2. GET /api/v2/jobs/{jobId}: возвращает статус задачи.

3. GET /api/v2/jobs/{jobId}/file: возвращает результат конверсии (файл)

4. GET /api/v2/jobs: возвращает список задач пользователя.

Клиентская часть разработана на TypeScript с использованием React. Проект организован в модули: компоненты, хуки, утилиты для API, типы. Основной компонент UploadSection реализует многошаговый процесс создания задачи и отображения списка задач. Для управления шагами используется простой конечный автомат на основе useState с тремя состояниями: UPLOAD (выбор файла), SELECT_FORMAT (выбор формата конвертации), CONFIRM (подтверждение и отправка). Переходы выполняются функцией moveForward. При подтверждении вызывается асинхронная функция createJob, которая отправляет файл и выбранный формат на сервер; после получения ответа новый объект задачи добавляется в состояние jobs, и процесс сбрасывается к началу.

Выбранный файл и формат хранятся в отдельных состояниях (selectedFile, selectedFormat). Для получения формата исходного файла используется утилита getInputFormat, извлекающая расширение и сопоставляющая его с допустимыми форматами. Компоненты FileDropSection и SelectFormatSection обеспечивают пользовательский интерфейс для каждого шага.

Отслеживание статусов созданных задач реализовано через периодический опрос. В useEffect с зависимостью от jobs запускается интервал (каждые 3 секунды), который собирает идентификаторы незавершённых задач и отправляет

запрос на эндпоинт fetchJobs. Полученные обновлённые статусы сливаются с текущим списком задач, после чего состояние обновляется. Интервал автоматически очищается при размонтировании компонента или изменении списка задач. Компонент JobListSection отображает список задач, их статусы и, при необходимости, ссылки на скачивание.

Такой подход обеспечивает реактивность интерфейса: пользователь видит изменения статусов в реальном времени без необходимости ручного обновления страницы. Конечный автомат для шагов создания задачи делает процесс интуитивно понятным и предотвращает некорректные последовательности действий.

Заключение

В ходе выполнения курсового проекта была достигнута поставленная цель – разработан асинхронный веб-сервис для конвертации медиафайлов (изображений, аудио, видео), позволяющий пользователям загружать файлы, немедленно получать идентификатор задачи и скачивать результат после её фоновой обработки.

В первой главе были изучены теоретические основы асинхронной обработки данных. Рассмотрено понятие асинхронности в веб-приложениях, выявлены её преимущества (отзывчивость интерфейса, эффективное использование ресурсов, масштабируемость, отказоустойчивость) и недостатки (усложнение архитектуры, дополнительные накладные расходы). Проанализированы способы организации очередей задач: реляционные базы данных, RabbitMQ и Apache Kafka. Проведён сравнительный анализ, показавший, что для условий проекта (слабый сервер, невысокая нагрузка) оптимальным выбором является использование PostgreSQL в качестве очереди. Изучены принципы работы S3-совместимых объектных хранилищ (MinIO, Ceph, SeaweedFS, Garage), что позволило обосновать выбор MinIO как наиболее лёгкого и простого в развёртывании решения. Также определён технологический стек реализации: Java 25, Spring Boot 4 для бэкенда и TypeScript, React для фронтенда.

Во второй главе спроектирована и реализована система. Разработана общая архитектура, определены компоненты и их взаимодействие. Спроектирована реляционная база данных (таблица conversion_jobs) и структура ключей в MinIO, обеспечивающие эффективное хранение метаданных и файлов. Реализован механизм асинхронной обработки на основе двухуровневой очереди (планировщик Spring и пул воркеров) с атомарным захватом задач через SELECT ... FOR UPDATE SKIP LOCKED, что гарантирует отсутствие состояния гонки. Разработаны REST API (создание задачи, получение статуса, скачивание результата, получение списка задач) и клиентский интерфейс на React с

конечным автоматом для управления состоянием и периодическим опросом статуса.

В ходе работы были выполнены все поставленные задачи:

1. Определено понятие асинхронности, выявлены сценарии применения и ключевые преимущества.

2. Выполнен сравнительный анализ брокеров сообщений и реляционных баз данных как очередей задач, сформулированы критерии выбора.

3. Изучены принципы работы и архитектура S3-совместимых хранилищ.

4. Проведён сравнительный анализ рассмотренных технологий и обоснован выбор конкретного стека технологий.

5. Разработана общая архитектура сервиса, определено взаимодействие компонентов.

6. Спроектирована схема базы данных и структура бакетов в объектном хранилище.

7. Реализован механизм асинхронной обработки задач (постановка в очередь, обработка воркерами, отслеживание статуса).

8. Выполнена реализация серверной части (REST API) и клиентского интерфейса.

Таким образом, все цели и задачи курсового проекта выполнены в полном объёме. Полученные результаты могут быть использованы в учебных целях, а также в реальных проектах, требующих асинхронной обработки медиафайлов в условиях ограниченных вычислительных ресурсов.

Список использованной литературы

1. Глоссарий // The Reactive Manifesto = Манифест реактивных систем : [сайт]. – 2025. – URL:https://reactivemanifesto.org/ru/glossary (дата обращения: 09.03.2026). – Текст: электронный.

2. Message Queue // RisingWave : [сайт]. – 2025. – URL:https://www.risingwave.com/glossary/message-queue (дата обращения: 13.03.2026).

3. Event-Driven Architecture (EDA) // RisingWave : [сайт]. – 2025. – URL:https://risingwave.com/glossary/event-driven-architecture-(eda) (дата обращения: 13.03.2026).

4. Urma, R.-G. Modern Java in Action / R.-G. Urma, M. Fusco, A. Mycroft. – Shelter Island: Manning Publications, 2019. – 592 с. – ISBN 9781617293566. – Текст: непосредственный.

5. Kafka: The Definitive Guide: Real-Time Data and Stream Processing at Scale / G. Shapira, T. Palino, R. Sivaram, K. Petty. – Sebastopol: O'Reilly Media, 2021. – 488 с. – ISBN 9781492043089. – Текст: непосредственный.

6. Cleary, S. Concurrency in C# Cookbook: Asynchronous, Parallel, and Multithreaded Programming / S. Cleary. – 2nd ed. – Sebastopol: O'Reilly Media, 2019. – 254 с. – ISBN 9781492054504. – Текст: непосредственный.

7. RabbitMQ Documentation / VMware Inc. – 2026. – Текст: электронный. – URL:https://www.rabbitmq.com/docs (дата обращения: 15.03.2026).

8. Gorelick, M. High Performance Python: Practical Performant Programming for Humans / M. Gorelick, I. Ozsvald. – Sebastopol: O'Reilly Media, 2014. – 370 с. – ISBN 978-1-449-36159-4. – Текст: электронный. – URL:https://pepa.holla.cz/wp-content/uploads/2016/08/High-Performance-Python.pdf (дата обращения: 15.03.2026).

9. Postgres CDC Without Debezium: Build SQS/Kafka in SQL // Stacksync : [блог компании]. – 2026. – 4 янв. – Текст: электронный. – URL:https://www.stacksync.com/blog/postgres-cdc-without-debezium-build-sqskafka-sql (дата обращения: 15.03.2026).

10. Оптимизация взаимодействия микросервисов с использованием асинхронных вызовов и брокеров сообщений (Kafka, RabbitMQ) / А. Э. Баженов, А. В. Райков, М. В. Нехаев, Н. В. Ореховский // Программные системы и вычислительные методы. – 2025. – № 4. – С. 77-93. – DOI 10.7256/2454-0714.2025.4.77228. – EDN URNSWS. – Текст: электронный. – URL:https://ru.aurora-journals.com/library_read_article.php?id=77228 (дата обращения: 16.03.2026).

11. Murty, J. Programming Amazon Web Services: S3, EC2, SQS, FPS, and SimpleDB / J. Murty. – Beijing; Sebastopol: O'Reilly Media, 2008. – 640 с. – ISBN 9780596515812. – Текст: непосредственный.